

torch.fft.ifft#

<https://pytorch.org/docs/stable/generated/torch.fft.ifft.html>

Description:

Computes the one dimensional inverse discrete Fourier transform of input. Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension. input (Tensor) – the input tensor n (int, optional) – Signal length. If given, the input will either be zero-padded or trimmed to this length before computing the IFFT.

Function Signature

```
torch.fft.ifft(input, n=None, dim=-1, norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> t = torch.tensor([ 6.+0.j, -2.+2.j, -2.+0.j, -2.-2.j])
>>> torch.fft.ifft(t)
tensor([0.+0.j, 1.+0.j, 2.+0.j, 3.+0.j])
```

Notes & Warnings

NOTE

Note Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension.

Detailed Documentation

Documentation

Computes the one dimensional inverse discrete Fourier transform of input.

Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension.

input (Tensor) – the input tensor

n (int, optional) – Signal length. If given, the input will either be zero-padded or trimmed to this length before computing the IFFT.

dim (int, optional) – The dimension along which to take the one dimensional IFFT.

norm (str, optional) –

Normalization mode. For the backward transform (ifft()), these correspond to:

"forward" - no normalization

"backward" - normalize by 1/n

"ortho" - normalize by 1/sqrt(n) (making the IFFT orthonormal)

Calling the forward transform (fft()) with the same normalization mode will apply an overall normalization of 1/n between the two transforms. This is required to make ifft() the exact inverse.

Default is "backward" (normalize by 1/n).

out (Tensor, optional) – the output tensor.